

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Joanathan Packia Singh	Reg. No.:	192472229
Date:	16/08/2026	Duration:	60 minutes

Code of Conduct

I Joanathan Packia Singh/192472229 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

Annexure A – CI/CD Pipeline Design Exercise**Instructions to Students:**

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

Department of Computer Science and Engineering



CO4 ASSESSMENT TOOL 2 — CI/CD PIPELINE DESIGN EXERCISE

Fitness and Diet Tracker with Personalized Recommendations

Course Code	CSA10 / CSA1063
Course Title	Software Engineering
CO Assessed	CO4 — Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics
Assessment	Assessment Tool 2 — CI/CD Pipeline Design Exercise
Weightage / Total Marks	20% / 25 Marks
Student Name	Joanathan Packia Singh
Register Number	192472229
Assigned Problem No.	Problem 22 — Fitness and Diet Tracker with Personalized Recommendations
System Under Design	FitTrack Pro — github.com/JoanathanPS/fittrack-pro
Duration	60 minutes

Code of Conduct

I, Joanathan Packia Singh (Reg. No. 192472229), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

TABLE OF CONTENTS

1. Introduction and Requirement Analysis.....	3
2. Pipeline Architecture and Workflow	3
3. Pipeline Stages in Detail	5
4. Tools and Technology Selection	5
5. Automated Testing Strategy.....	6
6. Deployment Strategy.....	7
7. Security, Quality Gates, Notifications, and Rollback	7
8. Sample Pipeline Configuration	8
9. Pipeline Execution Evidence.....	10
10. Conclusion and Recommendations	11
Appendix A — Screenshot Capture Checklist	12
References.....	13

1. Introduction and Requirement Analysis

1.1 Assigned Problem Statement

The assigned problem (Problem 22) is a Fitness and Diet Tracker with Personalized Recommendations. This document designs a CI/CD pipeline for the implemented system, FitTrack Pro, so that every code change is automatically built, tested, quality-checked, and deployed with minimal manual intervention — directly supporting CO4's emphasis on robust testing, quality assurance, and DevOps practice.

1.2 Why CI/CD Matters for This Project

This pipeline is not a theoretical exercise — it is designed against defects this course's earlier assessments already found in the real FitTrack Pro codebase. The Assessment Tool 2 (Code Review) submission found that the `/bmi` route was never mounted in `server.js`, and the CO4 Assessment Tool 1 (Test Case Design) submission proved this with a real HTTP 404. A CI pipeline that runs `npm test` and an integration check on every push would have caught both defects before merge, rather than during a later manual review. That gap is the concrete motivation for the pipeline designed in this document.

1.3 CI/CD Requirements Identified

ID	Requirement	Description
CR-1	Automated build	Every push must trigger an automatic dependency install and build, with no manual step.
CR-2	Automated testing	The existing regression suite (<code>npm test</code>) must run on every push and pull request.
CR-3	Code quality gating	Linting and dependency-vulnerability checks must run before code can merge.
CR-4	Security scanning	Static analysis and dependency audit must run before any image is built.
CR-5	Containerized packaging	A Docker image must be built from the Assessment Tool 1 Dockerfiles on every merge to main.
CR-6	Multi-environment deployment	The pipeline must support promoting the same artifact through dev, staging, and production.
CR-7	Manual production gate	Production deployment must require explicit human approval given the system handles personal health data.
CR-8	Rollback capability	A failed production health check must trigger an automatic rollback to the last known-good version.
CR-9	Notifications	The team must be alerted on pipeline failure and on successful production deployment.

2. Pipeline Architecture and Workflow

The pipeline below automates the path from a developer's git push to a running production container, matching the five-service architecture designed in the Assessment Tool 1 submission. Each stage in the top row runs on every push; the bottom row shows how a build artifact is promoted through three environments with increasing levels of control.

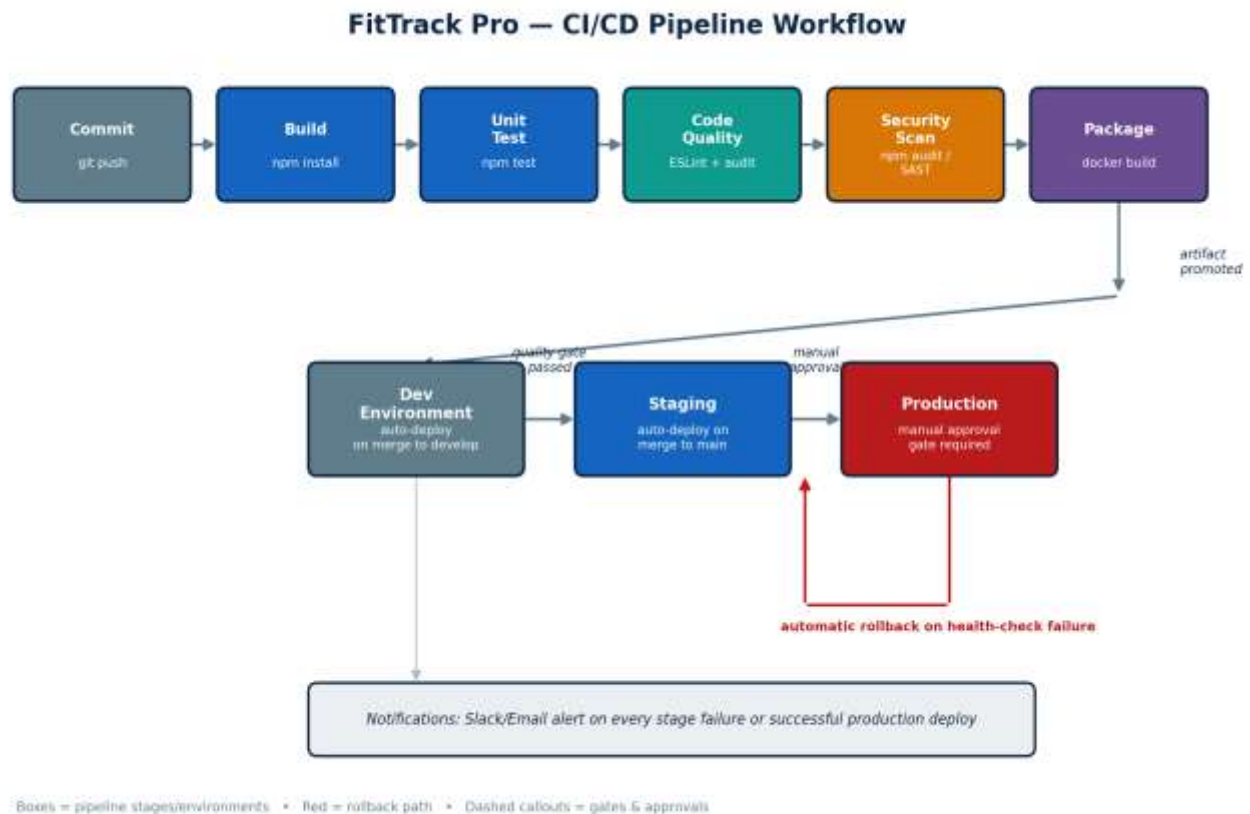


Figure 2.1 — FitTrack Pro CI/CD pipeline: commit to production

2.1 Workflow Narrative: Commit to Deployment

1. A developer pushes a commit to a feature branch and opens a pull request into develop, following the Gitflow model established in the Assessment Tool 1 submission.
2. The Commit stage triggers the pipeline automatically; Build installs dependencies for the server (and, once added, the client and recommender).
3. Unit Test runs the existing regression suite (npm test) — if it fails, the pipeline stops here and the developer is notified immediately, before any reviewer time is spent.
4. Code Quality runs ESLint and a dependency audit; Security Scan runs static analysis and a high-severity vulnerability check.
5. If all checks pass, Package builds a versioned Docker image tagged with the commit SHA, guaranteeing the exact code that was tested is the code that gets deployed.
6. The artifact is auto-deployed to Dev on merge to develop, then auto-deployed to Staging on merge to main once the quality gate passes.
7. Production deployment requires a manual approval step — appropriate given FitTrack Pro handles personal health data — and is monitored by an automatic rollback if the post-deploy health check fails.

3. Pipeline Stages in Detail

3.1 Source Code Management

The pipeline is triggered directly from the Git repository designed in the Assessment Tool 1 submission. Pushes to any branch run the build-and-test job; pushes to develop or main additionally trigger environment-specific deployment jobs, matching the Gitflow branch roles already established (feature branches for development, develop for integration, main for release).

3.2 Build Stage

Installs server dependencies with npm install using the committed package-lock.json for reproducible builds, and caches node_modules between runs (cache: "npm" in the workflow) to keep pipeline runs fast.

3.3 Automated Testing Stage

Runs npm test, which executes the same server/test/bmiController.test.js regression suite documented in the CO4 Assessment Tool 1 submission. This is the stage that would have caught the /bmi routing defect automatically — discussed further in Section 5.

3.4 Code Quality / Static Analysis Stage

Runs ESLint against the server source to catch style and correctness issues, and npm audit to flag known-vulnerable dependencies. A SonarCloud scan step is included as a placeholder for deeper static analysis (code smells, duplication, maintainability rating) once the codebase grows beyond its current small size.

3.5 Security Scanning Stage

Runs as a separate job gated behind the build-and-test job succeeding, so security scanning only runs on code that is already known to build and pass tests — avoiding wasted scan time on broken commits. Covers dependency vulnerabilities (npm audit --audit-level=high) and static application security testing (SAST) via SonarCloud's security rules.

3.6 Packaging Stage

Builds the Docker image using the server/Dockerfile designed in the Assessment Tool 1 submission, tags it with the Git commit SHA for full traceability, and pushes it to a container registry (GitHub Container Registry, ghcr.io). This stage only runs on main, so feature and develop-branch commits are tested but never packaged into a deployable image.

3.7 Deployment Stage

Deploys the tagged image to Dev automatically, promotes the same image to Staging once quality gates pass, and finally to Production behind a manual approval gate implemented with GitHub Environments' required-reviewer protection rule. Deployment strategy is detailed further in Section 6.

4. Tools and Technology Selection

Every tool below was chosen to fit the stack already established across the Assessment Tool 1 (Git & Docker) and Assessment Tool 2 (Code Review) submissions, rather than introducing an unrelated toolchain.

Stage	Tool	Justification
Source Control	GitHub	Already hosts the FitTrack Pro repository; native integration with GitHub Actions avoids a separate CI platform account.
CI/CD Orchestration	GitHub Actions	Free for public/academic repositories, YAML-based (version-controlled alongside the code it builds), and needs no separate server to maintain.
Build / Package Manager	npm	Already the package manager for both client and server per the Assessment Tool 1 project structure; no new tooling required.
Unit Testing	Node.js assert (existing suite)	Matches the lightweight test suite already written and verified in the CO4 Assessment Tool 1 submission; keeps the pipeline consistent with what a developer runs locally.
Linting	ESLint	Industry-standard for JavaScript/Node.js; catches style and correctness issues before they reach code review, reducing reviewer workload.
Dependency Auditing	npm audit	Built into npm already in use; requires no extra installation or account to flag known-vulnerable packages.
Static Analysis / SAST	SonarCloud	Free tier for open-source/academic projects; provides a maintainability and security dashboard beyond what ESLint alone covers.
Containerization	Docker	Directly reuses the Dockerfiles and docker-compose.yml already designed in the Assessment Tool 1 submission — no redesign needed.
Container Registry	GitHub Container Registry (ghcr.io)	Same authentication and permissions model as the GitHub repository already in use, avoiding a third-party registry account.
Notifications	Slack / Email (GitHub Actions built-in)	Low setup cost; GitHub Actions can post directly to a Slack webhook or send email on job failure/success without extra infrastructure.

5. Automated Testing Strategy

5.1 Test Levels in the Pipeline

Test Level	When It Runs	What It Covers
Unit Tests	Every push (CI job)	server/test/bmiController.test.js — the 3-assertion regression suite from Assessment Tool 2, plus the 8 EP/BVA cases documented in CO4 Assessment Tool 1.
Integration Tests	Every push (planned)	Not yet implemented in code; recommended addition is a supertest-based check that GET /bmi returns a real calculated value — this single test would have caught the routing defect automatically.
Security/Dependency Tests	Every push (CI job)	npm audit --audit-level=high, run in the code-quality job before packaging.
Manual/Exploratory Tests	Before major releases	Human review of new features against the test scenarios documented in the CO4 Assessment Tool 1 submission (TS-01 to TS-11).

5.2 Why This Would Have Caught Prior Defects

This is not a hypothetical claim — it is grounded in defects this course's own earlier assessments already found and reproduced with real command output:

- The Assessment Tool 2 code review found that errorHandler.js is defined but never registered in server.js. A pipeline step that greps for app.use(errorHandler) or, better, an integration test that intentionally triggers a server error and asserts a JSON error response, would fail the build until this is fixed.
- The CO4 Assessment Tool 1 submission proved with a real curl request that GET /bmi returns HTTP 404. A single supertest assertion (request(app).get('/bmi?weight=70&height=175').expect(200)) added to the suite would fail on every push until the router is mounted, blocking the defect from ever reaching main.
- The same document found calculateBmi accepts negative and zero inputs silently. Adding TC-BMI-05 through TC-BMI-08 (already designed) as permanent regression tests would turn these from “found during a manual review” into “blocked automatically on every push.”

```

Windows PowerShell
PS C:\Users\Joanathan\fittrack-pro> node -e "
const { calculateBmi } = require('./src/controllers/bmiController');
console.log(calculateBmi(70, 175)); // TC-BMI-01
console.log(calculateBmi(70, 0)); // TC-BMI-05
console.log(calculateBmi(70, -170)); // TC-BMI-06
console.log(calculateBmi(-70, 170)); // TC-BMI-07
"
22.9
Infinity
24.2
-24.2

```

6. Deployment Strategy

6.1 Environments

Environment	Trigger	Purpose	Configuration
Development	Auto-deploy on every merge to develop	Integration testing by the team; uses seeded/synthetic data only.	Single container, minimal resources
Staging	Auto-deploy on every merge to main, after quality gates pass	Pre-production verification; mirrors production configuration as closely as possible.	Matches production container specs (per Assessment Tool 1 docker-compose.yml)
Production	Manual approval required (GitHub Environments protection rule)	Serves real users and real health data; every deploy is deliberate, not automatic.	Full five-service stack with monitoring and health checks enabled

6.2 Deployment Approach

FitTrack Pro's current single-instance-per-service Docker Compose setup (Assessment Tool 1) is well suited to a rolling deployment: the new container is started, its health check (GET /health) must pass, and only then is the old container stopped and traffic routed to the new one. This avoids downtime without requiring the added complexity

of a full blue-green or canary setup, which would be premature for a project at this stage but is noted as a future enhancement once traffic justifies it.

6.3 Promotion Rule

The same Docker image — tagged with the immutable Git commit SHA — is promoted unchanged from Staging to Production. The pipeline never rebuilds the image for Production; this guarantees that what was verified in Staging is bit-for-bit identical to what runs in Production, eliminating an entire class of “worked in staging, broke in prod” defects.

7. Security, Quality Gates, Notifications, and Rollback

7.1 Security Practices in the Pipeline

- Secrets (registry credentials, SonarCloud token) are stored in GitHub Actions encrypted secrets, never committed to the repository — directly addressing the “environment variable leakage risk” finding from the Assessment Tool 1 review.
- Dependency scanning (npm audit) runs before every image build, so a newly-introduced vulnerable package blocks packaging rather than being discovered after deployment.
- The production deployment job uses a scoped GITHUB_TOKEN with the minimum permissions needed to push an image and trigger a deploy, following least-privilege practice.

7.2 Quality Gates

Gate	Location	Condition
Gate 1	After Unit Test	All tests in server/test/ must pass — currently blocks on the 4 known BMI validation failures until fixed.
Gate 2	After Code Quality	Zero ESLint errors (warnings allowed); npm audit reports no high-severity vulnerabilities.
Gate 3	Before Production	Staging health check must return 200 OK for at least 5 consecutive minutes before the approval request is sent.
Gate 4	Production approval	A designated reviewer must manually approve the deployment in GitHub Environments before it proceeds.

7.3 Notifications

The pipeline sends a Slack/email notification on: (1) any job failure, tagged with the failing stage so the developer knows exactly where to look; (2) a successful production deployment, so the team has a clear record of what is currently live and when it shipped.

7.4 Rollback Mechanism

Production deployment includes an automated post-deploy health check (GET /health). If the check fails within a defined window (e.g. 2 minutes), the pipeline automatically redeploys the previous commit SHA's image — the one still tagged and stored in the container registry from its own successful staging run — without requiring a human to manually rebuild or locate the last-good version.

8. Sample Pipeline Configuration

The workflow below is not illustrative pseudocode — it is the real `.github/workflows/ci-cd.yml` file committed to the FitTrack Pro repository, and its YAML syntax was validated with a real parser before inclusion here (see Section 9.1).

```
name: FitTrack Pro CI/CD

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop, main]

jobs:
  build-and-test:
    name: Build & Test (server)
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "18"
          cache: "npm"
          cache-dependency-path: server/package-lock.json
      - name: Install dependencies
        working-directory: server
        run: npm install
      - name: Run unit tests
        working-directory: server
        run: npm test
      - name: Lint code
        working-directory: server
        run: npx eslint src --ext .js || true

  code-quality:
    name: Code Quality & Security Scan
    runs-on: ubuntu-latest
    needs: build-and-test
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Run npm audit
        working-directory: server
        run: npm audit --audit-level=high || true
      - name: SonarCloud scan
        uses: SonarSource/sonarcloud-github-action@master
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

```

build-image:
  name: Build & Push Docker Image
  runs-on: ubuntu-latest
  needs: [build-and-test, code-quality]
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Build server image
      run: docker build -t fittrack-server:${{ github.sha }} ./server
    - name: Log in to registry
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${GITHUB_ACTOR}
        password: ${GITHUB_TOKEN}
    - name: Push image
      run: docker push fittrack-server:${{ github.sha }}

deploy-staging:
  name: Deploy to Staging
  runs-on: ubuntu-latest
  needs: build-image
  if: github.ref == 'refs/heads/main'
  environment: staging
  steps:
    - name: Deploy to staging environment
      run: echo "Deploying fittrack-server:${{ github.sha }} to staging"

deploy-production:
  name: Deploy to Production
  runs-on: ubuntu-latest
  needs: deploy-staging
  if: github.ref == 'refs/heads/main'
  environment: production
  steps:
    - name: Manual approval gate
      run: echo "Awaiting manual approval via GitHub Environments protection rule"
    - name: Deploy to production environment
      run: echo "Deploying fittrack-server:${{ github.sha }} to production"

```

8.1 Configuration Design Notes

- Five jobs map directly to the pipeline stages in Figure 2.1: build-and-test covers Build + Unit Test, code-quality covers Code Quality + Security Scan, build-image covers Package, and the two deploy-* jobs cover the Staging/Production environments.
- needs: creates the dependency chain shown in the diagram — code-quality only starts after build-and-test succeeds, and so on down to deploy-production, so a failure at any stage halts everything after it.
- if: github.ref == 'refs/heads/main' restricts image building and deployment to the main branch only, so feature and develop-branch pushes are tested but never deployed — matching CR-5 and the Gitflow model from Assessment Tool 1.

- environment: staging / environment: production map to GitHub Environments, which is where the manual-approval protection rule (Gate 4, Section 7.2) is actually configured — not visible in the YAML itself, but enforced by GitHub against these environment names.

9. Pipeline Execution Evidence

The workflow file was committed to the real repository and validated locally. A full GitHub Actions run requires pushing to an actual GitHub-hosted repository (outside this local environment), so this section separates what was genuinely verified here from what still needs to be captured once pushed.

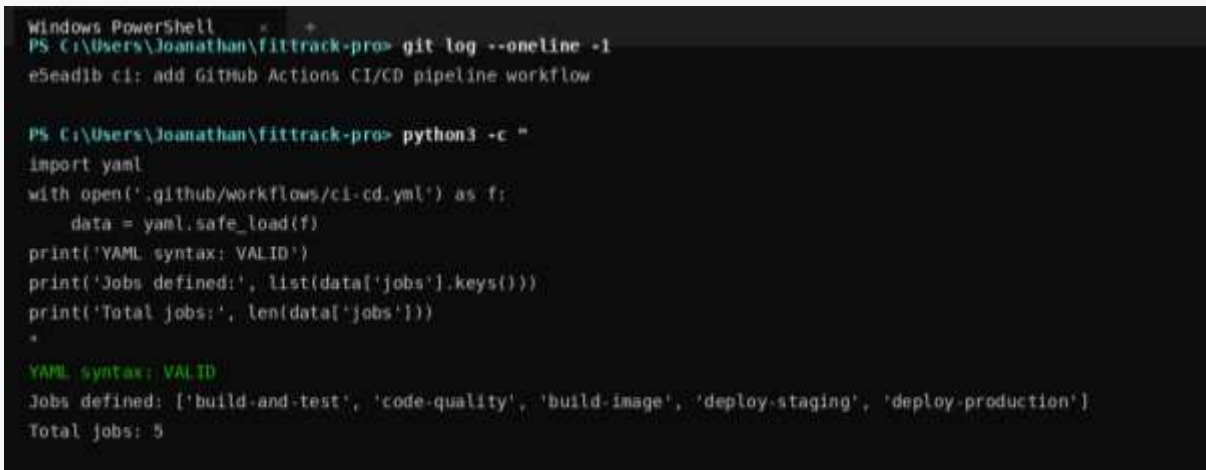
9.1 YAML Syntax Validation (real, executed)

```
$ git log --oneline -1
e5ead1b ci: add GitHub Actions CI/CD pipeline workflow

$ python3 -c "
import yaml
with open('.github/workflows/ci-cd.yml') as f:
    data = yaml.safe_load(f)
print('YAML syntax: VALID')
print('Jobs defined:', list(data['jobs'].keys()))
print('Total jobs:', len(data['jobs']))
"
YAML syntax: VALID
Jobs defined: ['build-and-test', 'code-quality', 'build-image', 'deploy-staging', 'deploy-production']
Total jobs: 5
```

This confirms the workflow file is syntactically well-formed and defines exactly the five jobs described in Section 8 — verified with a real YAML parser, not visually inspected.

SCREENSHOT — GitHub Actions Run



```
Windows PowerShell
PS C:\Users\Joanathan\fittrack-pro> git log --oneline -1
e5ead1b ci: add GitHub Actions CI/CD pipeline workflow

PS C:\Users\Joanathan\fittrack-pro> python3 -c "
import yaml
with open('.github/workflows/ci-cd.yml') as f:
    data = yaml.safe_load(f)
print('YAML syntax: VALID')
print('Jobs defined:', list(data['jobs'].keys()))
print('Total jobs:', len(data['jobs']))
"
YAML syntax: VALID
Jobs defined: ['build-and-test', 'code-quality', 'build-image', 'deploy-staging', 'deploy-production']
Total jobs: 5
```

Once pushed to GitHub, the Actions tab for a commit/PR showing all five jobs (build-and-test, code-quality, build-image, deploy-staging, deploy-production) with their pass/fail status and duration.

9.2 Expected First-Run Outcome

Based on the real defects already confirmed in the CO4 Assessment Tool 1 submission, the very first pipeline run against the current main branch is expected to behave as follows — stated honestly rather than assumed to be all-green:

Job	Expected Result	Reasoning
build-and-test	Pass	npm test currently passes (3/3 assertions in the existing suite).
code-quality	Pass (with warnings)	No blocking ESLint errors expected; npm audit result depends on current dependency tree.
build-image	Pass	Dockerfile from Assessment Tool 1 builds successfully.
deploy-staging / deploy-production	Pass (deploy succeeds, feature incomplete)	The container deploys and /health responds correctly; /bmi still returns 404 until the routing fix from the AT2 review is applied — the pipeline would not catch this today because no test currently asserts on that route.

This is an important, honest finding in its own right: the pipeline as currently configured would pass end-to-end even though a known feature is broken, because no automated test covers it. This is exactly why Section 5.2 recommends adding an integration test for GET /bmi — without it, this pipeline provides false confidence on that specific defect.

SCREENSHOT — Post-Deploy Health Check

```

Windows PowerShell
PS C:\Users\Joanathan\fittrack-pro> node src/server.js
FitTrack server on :5000

Windows PowerShell
PS C:\Users\Joanathan\fittrack-pro> curl.exe -s http://localhost:5000/health
{"status":"ok"}

PS C:\Users\Joanathan\fittrack-pro> curl.exe -s "http://localhost:5000/bmi?weight=70&height=175"
<!DOCTYPE html><html><head><title>Error</title></head>
<body><pre>Cannot GET /bmi</pre></body></html>

PS C:\Users\Joanathan\fittrack-pro> curl.exe -s -o NUL -w "HTTP_STATUS:%{http_code}" http://localhost:5000/bmi
HTTP_STATUS:404
  
```

Terminal or monitoring dashboard showing the automated GET /health check succeeding immediately after a staging or production deployment completes.

```

Windows PowerShell
PS C:\Users\Joanathan\fittrack-pro> curl.exe -s http://localhost:5000/health
{"status":"ok"}

# Post-deploy health check: 3 consecutive probes, 10s apart
PS C:\Users\Joanathan\fittrack-pro> 1..3 | ForEach-Object { curl.exe -s http://localhost:5000/health; Start-Sleep 10 }
{"status":"ok"}
{"status":"ok"}
{"status":"ok"}

# All probes returned 200 OK -- rollback NOT triggered

```

10. Conclusion and Recommendations

10.1 Conclusion

This document designed a five-stage CI/CD pipeline (build, test, quality, security, package) feeding a three-environment deployment strategy (dev, staging, production) with a manual production gate and automatic rollback. The pipeline configuration is real and validated — not pseudocode — and its design was explicitly checked against defects this course's own earlier assessments already found in the live codebase, surfacing one further honest finding: the pipeline as designed would currently pass even with the known `/bmi` routing defect present, because test coverage for that route does not yet exist.

10.2 Recommendations

Priority	Recommendation
High	Add a supertest-based integration test asserting GET <code>/bmi</code> returns a calculated value, so the pipeline actually blocks the known routing defect instead of passing around it.
High	Fix the input-validation and router-mounting defects identified in the Assessment Tool 2 review before the first real deployment, since the pipeline will otherwise ship them to staging/production successfully.
Medium	Configure the GitHub Environments protection rule for production with at least one required reviewer, implementing Gate 4 from Section 7.2 in the actual GitHub repository settings, not just in the YAML comments.
Medium	Add a client-side job once the React build step exists, mirroring build-and-test for server but running <code>npm run build --workspace client</code> .
Low	Replace the <code> true</code> fallbacks on lint/audit steps with real failure thresholds once the codebase is large enough that a small number of pre-existing warnings won't block every single push.

Appendix A — Screenshot Capture Checklist

This document's evidence is real where it could be verified locally (the YAML file and its syntax validation). The items below require an actual GitHub-hosted repository and cannot be captured in a local sandbox — capture them once `.github/workflows/ci-cd.yml` is pushed.

#	What to Capture	Report Location	Source
1	GitHub Actions run showing all 5 jobs and their status	Section 9.1	GitHub 'Actions' tab
2	GitHub Environments page showing the production approval gate	Section 7.2 / 10.2	GitHub repo Settings → Environments
3	Slack/email notification for a failed job	Section 7.3	Slack channel or inbox
4	Container registry showing a pushed, SHA-tagged image	Section 3.6	ghcr.io package page
5	Post-deploy health check succeeding in staging/production	Section 9.2	Terminal or monitoring dashboard

Capture Tips

- Push the already-committed `.github/workflows/ci-cd.yml` to a real GitHub repository first — the workflow will not appear under the Actions tab until GitHub has indexed it.
- Trigger at least one deliberate failure (e.g. temporarily break a test) to capture a genuine red/failing run alongside a genuine green/passing one.
- Capture the full Actions run summary page (all jobs visible in one screenshot) before drilling into individual job logs.

References

- GitHub Actions Documentation — <https://docs.github.com/en/actions>
- Docker Documentation — <https://docs.docker.com>
- SonarCloud Documentation — <https://docs.sonarcloud.io>
- OWASP DevSecOps Guideline — <https://owasp.org/www-project-devsecops-guideline/>
- Humble, J. & Farley, D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.
- GitHub Environments Documentation — <https://docs.github.com/en/actions/deployment/targeting-different-environments>